

TEMPORAL GRAPHS: PARAMETERS AND GAMES

Jess Enright, Sam Hand, Laura Larios-Jones, Tom Davot, Jayakrishnan Madathil, Kitty Meeks, many others

12 November 2026

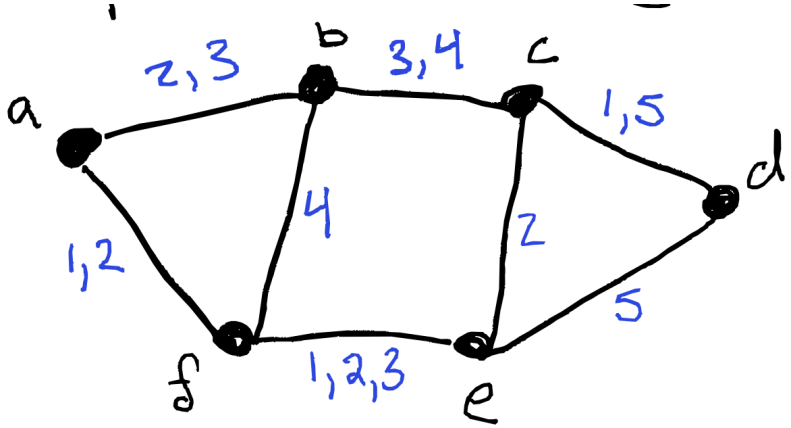
School of Computing Science, University of Glasgow

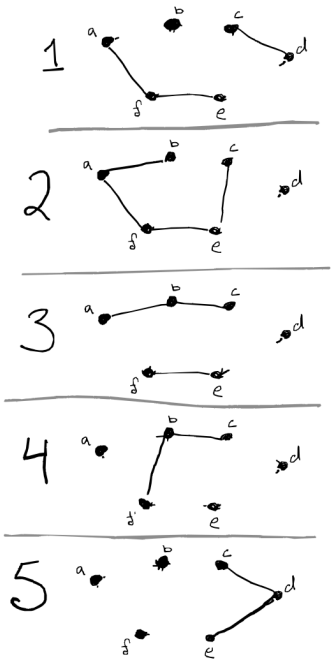
The Plan

- Temporal graphs
- … and what I'm not talking about
- … and why I got interested in them
- Problems on temporal graphs
- A crash course in parameterised algorithmics
- Parameters for temporal graphs
- Games: pursuit and burning

TEMPORAL GRAPHS

Here, a temporal graph is a graph with a function that specifies when an edge is active: $\mathcal{G} = (G, \lambda)$





A few important ideas:

- The underlying graph, or footprint
- lifetime

EVERYTHING IS HARDER ON TEMPORAL GRAPHS

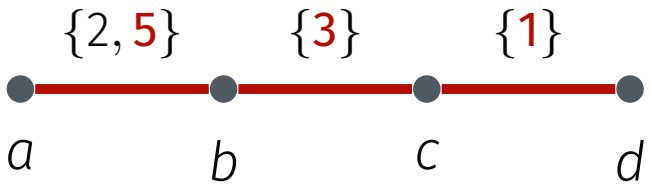
MAXIMUM TEMPORAL MATCHING

Input: A temporal graph (G, λ) and $\Delta \in \mathbb{N}$.

Question: What is the maximum cardinality of a set of time-edges $\mathcal{E}$ such that, for every pair of time-edges (wx, t_1) and (yz, t_2) in $\mathcal{E}$, either $\{w, x\} \cap \{y, z\} = \emptyset$ or $|t_1 - t_2| \geq \Delta$.



A maximum matching in the footprint: **2 edges**



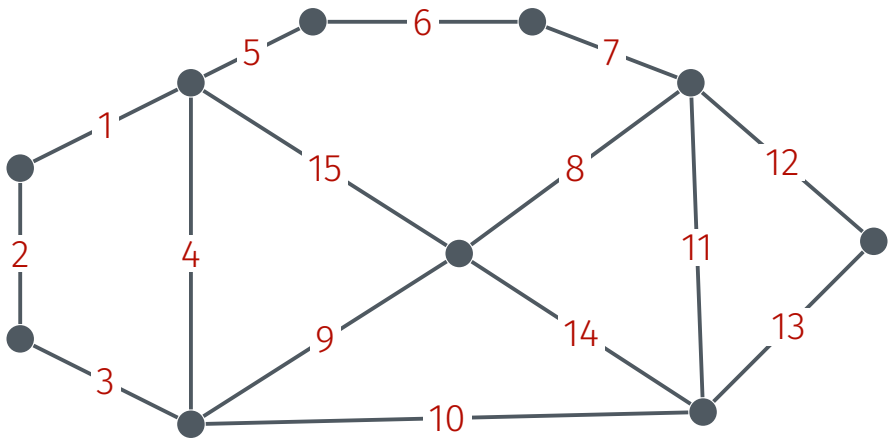
A maximum Δ-temporal matching, $\Delta = 2$: **3 time-edges**

b and c are each matched twice, because their two appearances are Δ apart. But we had to pick the *right* appearance of ab: taking $(ab, 2)$ instead of $(ab, 5)$ would clash with $(bc, 3)$.

Everything is harder on temporal graphs

Eulerian Circuit (the static problem)
Input: A connected graph G.
Question: Is there a closed walk using every edge exactly once?

Euler, 1736: yes iff every vertex has **even degree**, and Hierholzer finds one in linear time.



Every degree here is 2 or 4: the numbers give one Eulerian circuit.

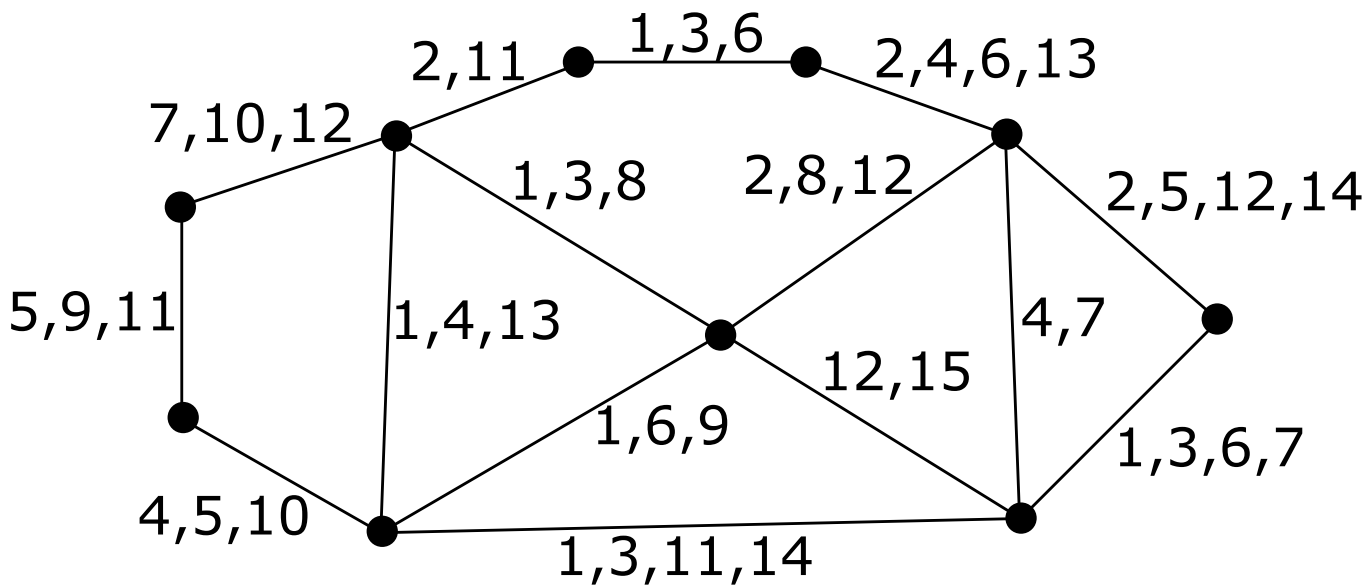
Everything is harder on temporal graphs

TempEuler

Input: A temporal graph (G, λ).

Question: Is there a closed temporal walk that contains every edge of G exactly once?

Note: this is also called Eulerian Trail by Marino & Silva.

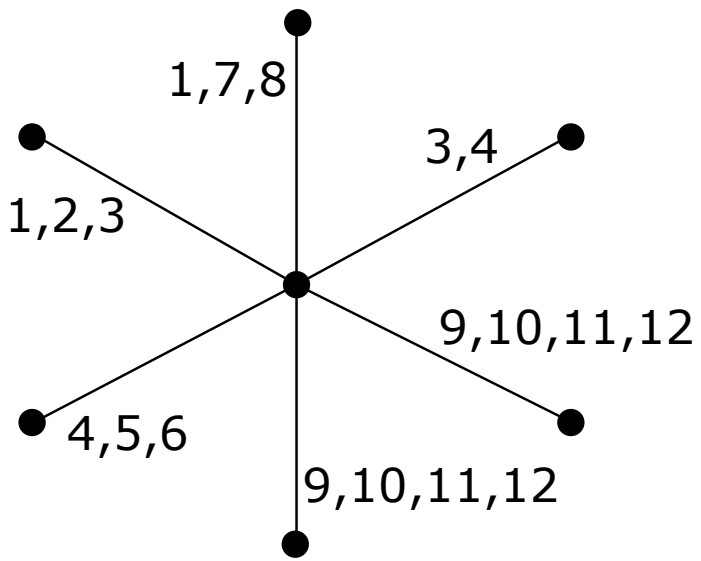


EVERYTHING IS HARDER ON TEMPORAL GRAPHS

StarExp

Input: A temporal graph (S_n, λ), where S_n is a star with n leaves.

Question: Does there exist a temporal walk, starting and ending at the centre of the star, that visits every vertex?

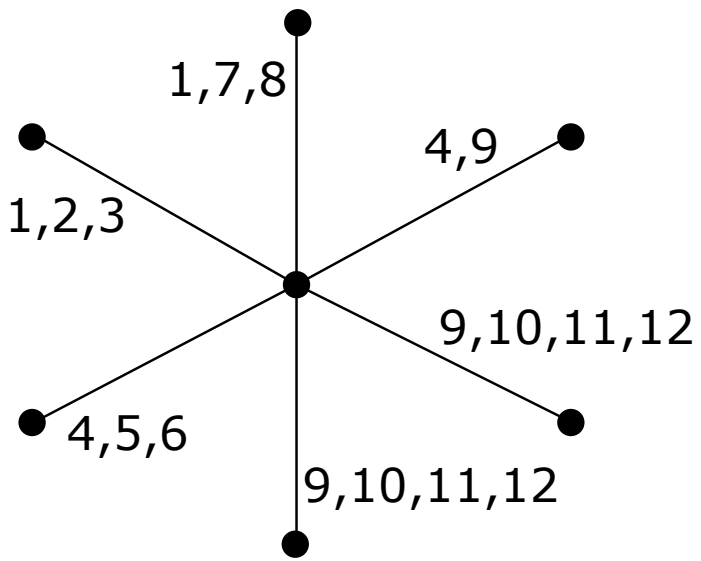


EVERYTHING IS HARDER ON TEMPORAL GRAPHS

StarExp

Input: A temporal graph (S_n, λ), where S_n is a star with n leaves.

Question: Does there exist a temporal walk, starting and ending at the centre of the star, that visits every vertex?



PROBLEMS ON TEMPORAL GRAPHS

It's not just hard, but complicated - many problems on static graphs can be made temporal - but often in lots of different ways!

Example: Does there exist a **temporal clique** of size k?

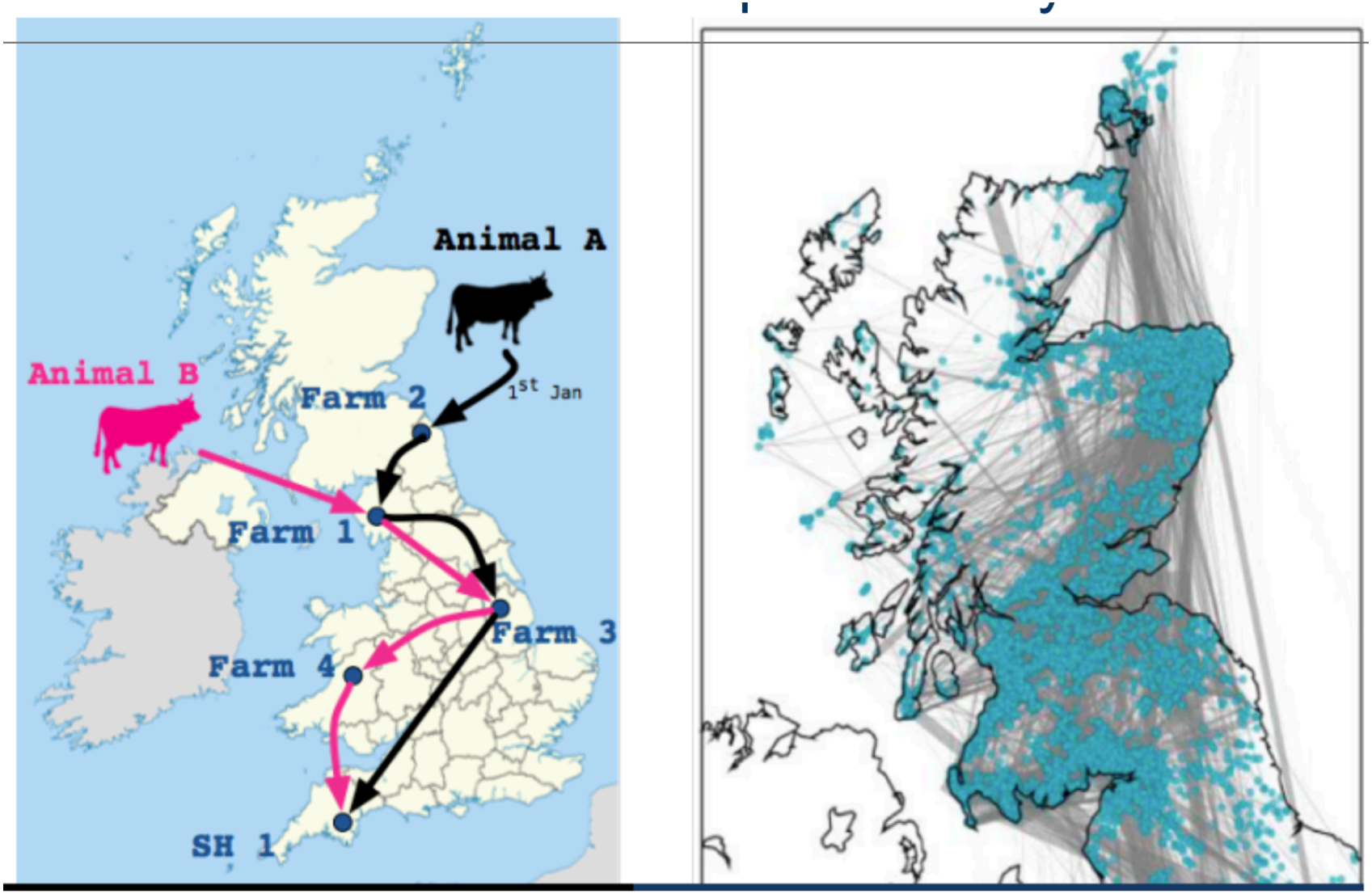
- What counts for adjacency?
  - **ever** adjacent?
  - **always** adjacent?
  - all adjacent at the **same time**
  - adjacent within some **time window** or with some **frequency**?

Depending on your definitions, the same vertex set could be both a temporal clique and a temporal independent set!

So: why are you ruining your life (and the lives of your students) with these, Jess?

- Fun
- ~~Profit~~ Applications

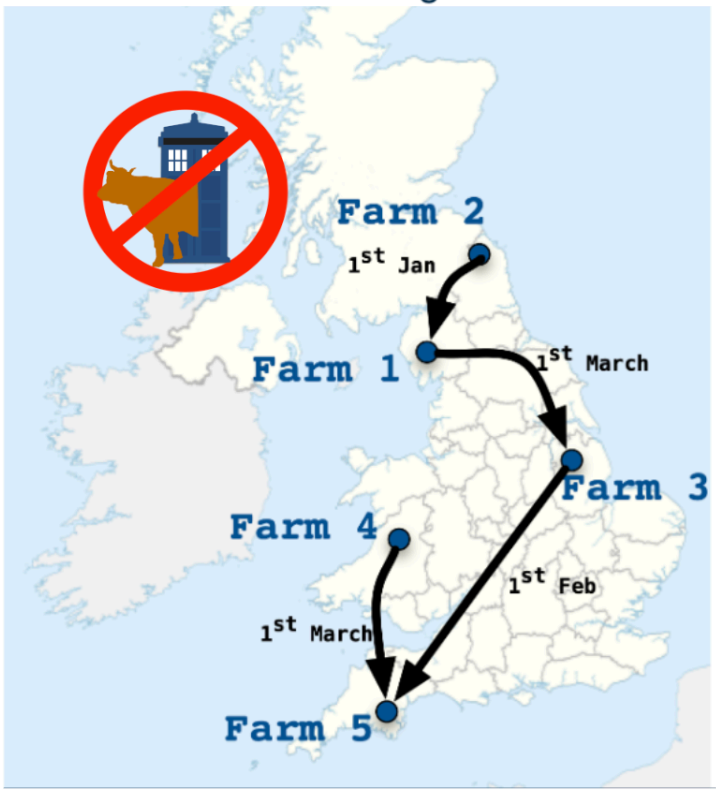
IT ALL STARTED WITH COWS

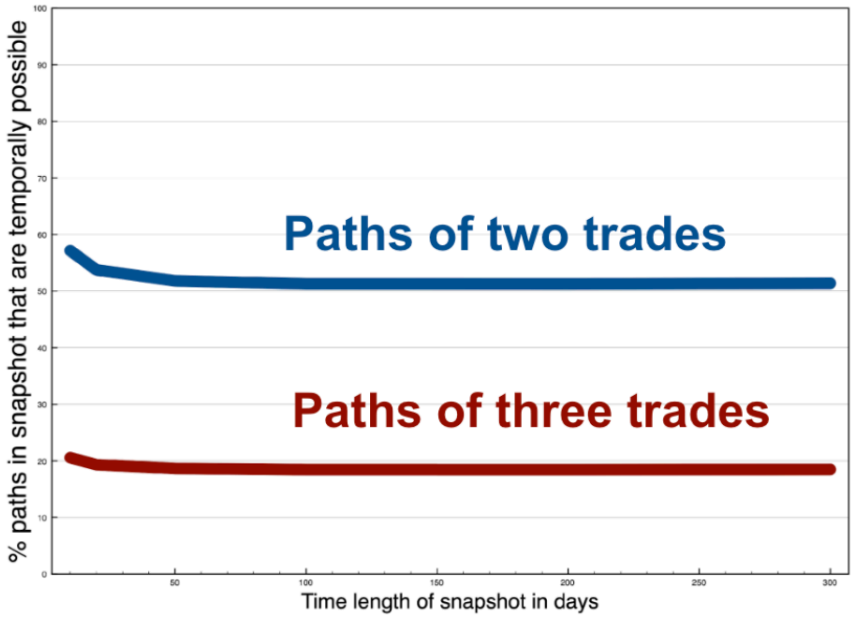


DISEASES AND CATTLE TRAVEL ONLY FORWARD IN TIME

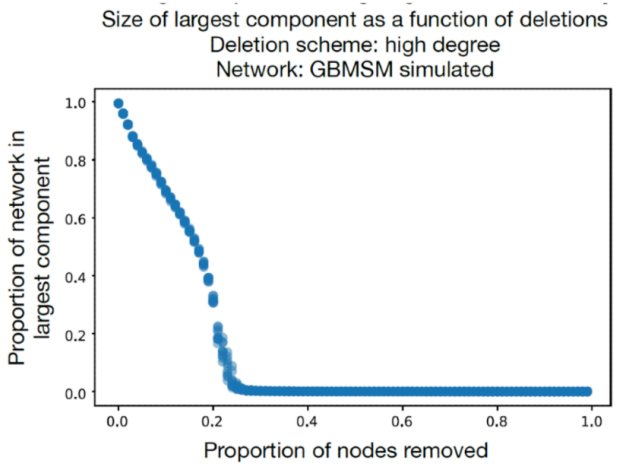
If we **ignore** the timing of movements: There is a path of three hops from Farm 2 to Farm 5

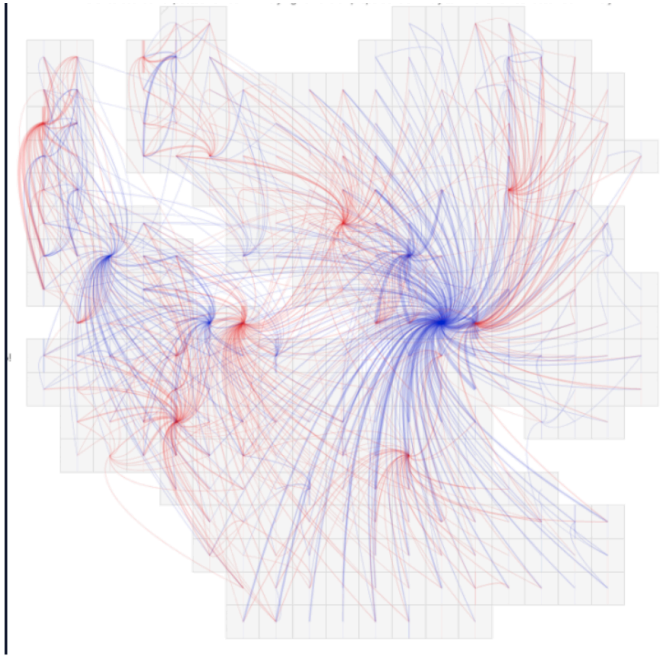
If we **consider** the timing of movements: There is **no** path from Farm 2 to Farm 5





HUMANS GET DISEASES TOO





For both fun and profit we want to solve hard problems on temporal graphs

What do you do with an NP-hard problem?

Suppose you want to solve one of these, and you want to solve it **exactly**.

- ~~Give up~~
- Approximate it - but this may be impossible or insufficient
- Heuristics - often excellent in practice, but no guarantees
- Use a solver and accept an exponential in n - fine until n is the size of the cattle trade network

Parameterised approach: work out what it is about your instances that makes them hard and tolerate an exponential related to that if it is small.

An instance is now a **pair** (I, k): the input, plus a *parameter* - some number we hope is small.

A problem is fixed-parameter tractable (FPT) if it can be solved in time

$$f(k) \cdot |I|^{\mathcal{O}(1)}$$

for some computable f. All of the badness is confined to f.

Compare **XP**: time $|I|^{f(k)}$, e.g. the n^k you get by trying every k-subset.

Running time	$n = 10^4$, $k = 20$
$2^k \cdot n$	about 10^{10} steps: fine
n^k	about 10^{80} steps: not fine

THE CLASSIC EXAMPLE

VERTEX COVER

Input: A graph G and an integer k.

Question: Is there a set of at most k vertices meeting every edge?

NP-complete, but: pick any edge uv; every cover contains u or v. Branch on which, and recurse with $k - 1$.

A search tree of depth k and branching factor 2 gives $\mathcal{O}(2^k(n + m))$.

So a 300-vertex network with a cover of size 20 is entirely reasonable, even though the problem is NP-complete.

WHERE DOES THE PARAMETER COME FROM?

Two broad flavours:

- **Solution size** - k is the thing you are asked for: the size of the cover, the number of deletions, the length of the walk.
- **Structural width** - k measures how far the input is from something we can already handle: treewidth, vertex cover number, neighbourhood diversity, cliquewidth.

A parameter earns its keep if:

- it is genuinely **small** on the instances we care about,
- we can **compute** (or approximate) it, and
- bounding it actually **buys** us an algorithm.

The first of these is an empirical question - we will come back to it at the end.

SO WE NEED PARAMETERISED APPROACHES!

How might we parameterise?

- Restrict the underlying graph
- Restrict the temporal structure
- Restrict something else: e.g. solution size

RESTRICTING THE UNDERLYING STRUCTURE

- Maximum Temporal Matching is NP-hard even when G is a path (Mertzios, Molter, Niedermeier, Zamaraev & Zschoche, 2020).
- TempEuler is NP-complete even if:
  - each edge appears at only 2 times (Marino & Silva, 2021);
  - each edge appears at only 3 times, and G has feedback vertex number one (Bumpus & Meeks, 2021);
  - each edge appears at only 4 times, and G has vertex cover number 2 (Bumpus & Meeks, 2021).
- StarExp is solvable in polynomial time if each edge appears at most 3 times (Akrida, Mertzios & Spirakis, 2019), but NP-complete if edges are allowed to appear 4 or more times (Bumpus & Meeks, 2021); G is always a tree of vertex-cover number 1.

TEMPORAL AND UNDERLYING STRUCTURE

A sampler:

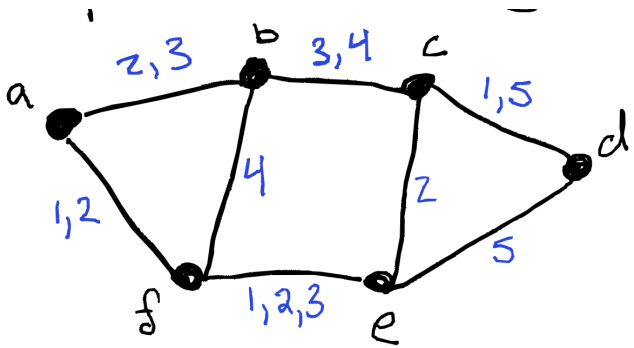
- lifetime
- lifetime plus underlying treewidth
- various temporal interpretations of treewidth
- timed vertex cover notions
- limiting the number of things you need to think about at a time
  - edge interval membership width
  - **vertex interval membership (VIM) width**
  - TIM-width
- Three nested ones that work for dense graphs:
  - **temporal neighbourhood diversity**
  - temporal modular width
  - temporal cliquewidth
- temporal clustering closedness

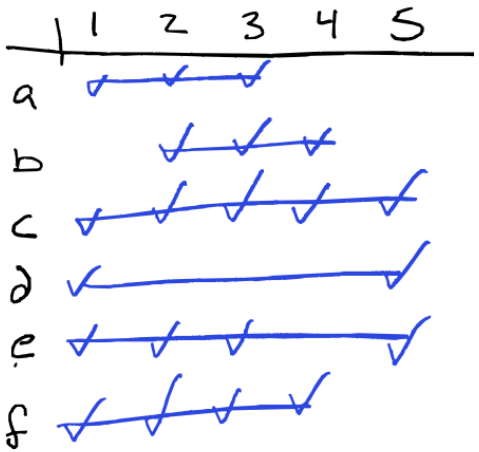
TEMPORAL PARAMETERS

A very restrictive temporal parameter:

~~vertex-interval-membership~~ VIM width

If a temporal graph has low VIM width that at every time most vertices have either finished being active or have not yet been active.



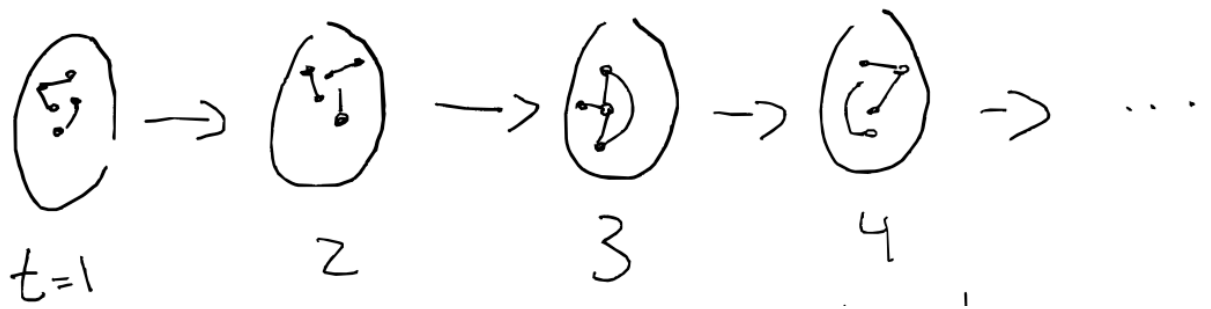


Why do we care about this? Sometimes we can devise algorithms that are $O(poly(n) \cdot f(\omega))$ where ω is our width

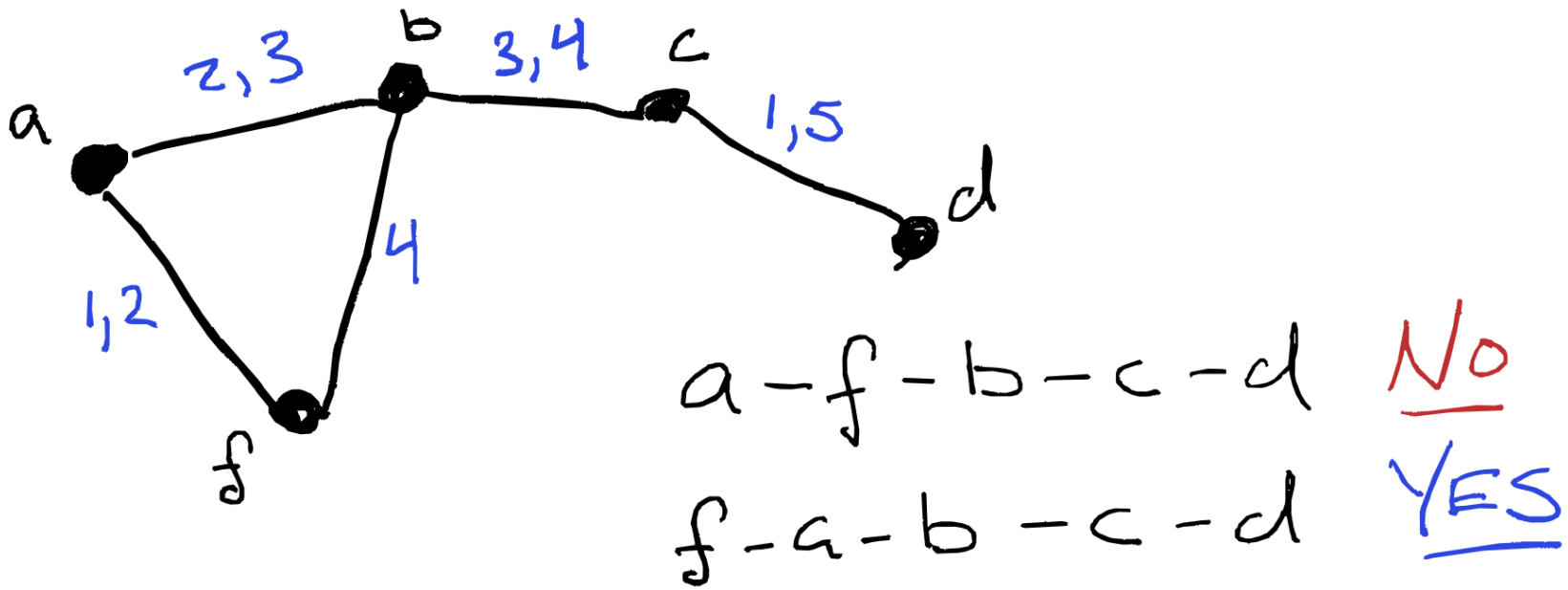
VIM DECOMPOSITION DPS

How do VIM-decomposition dynamic programs work?

- make a series of bags, one for each time
- starting at the beginning of time, consider all possibilities at each bag
  - and be able to check that a possibility is consistent with one at the previous time, and is acceptable
- get to the end of time, and see if you have solved the problem



Example: **Temporal Hamiltonian Path**



Is there a strict temporal path that visits every vertex exactly once?

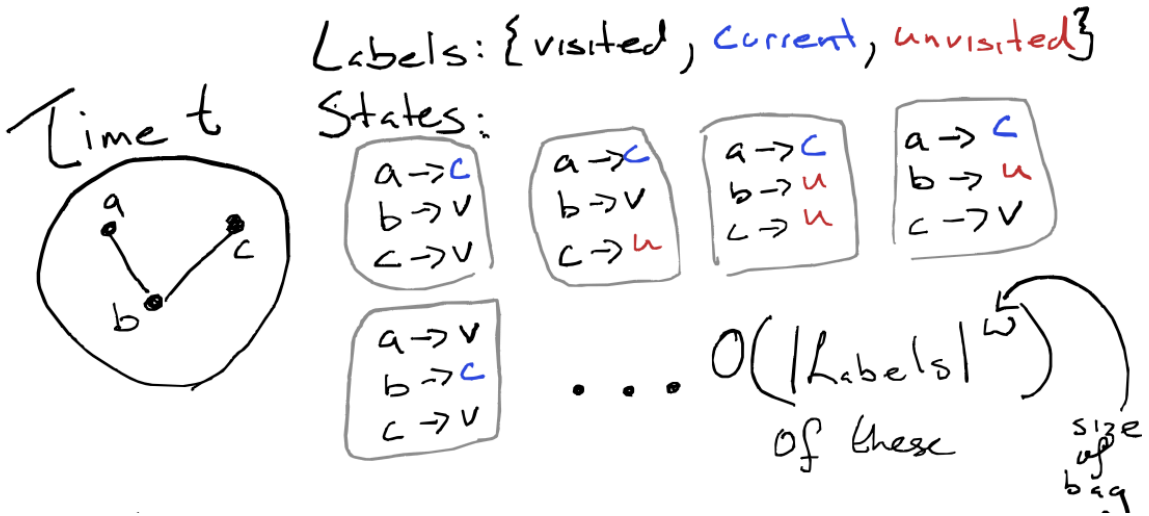
This is of course hard from the static case.

EX: TEMPORAL HAMILTONIAN PATH

At a bag, a state must record which vertices are visited already, where the path ends so far.

Key ingredients:

- we can generate all the states
- we can check if a state is consistent with previous state
- our state captures everything we need



Much as I love these DPs they do start to feel a little similar.

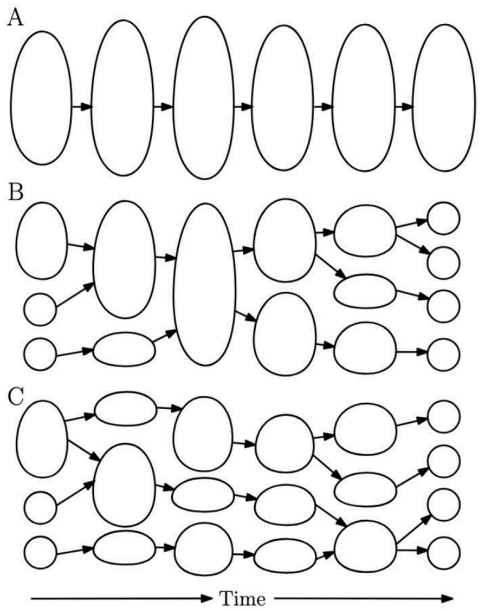
This has inspired a meta-algorithm that gives VIM DPs on a well-behaved type of problem.

What ingredients did we need?

- states that capture the problem, and we can ignore vertices outside their active interval
- ability to generate starting states
- ability to check states from time to time
- ability to check state at end-of-time to show we have solved problem

BUT THERE'S MORE!

~~Tree Interval Membership~~ TIM width



Here we need to be able to ignore not just vertices that are done being active or not yet active, but also ones that are in a different component in a particular way.

TIM IS MORE COMPLICATED

Sometimes TIM is more useful, sometimes VIM.

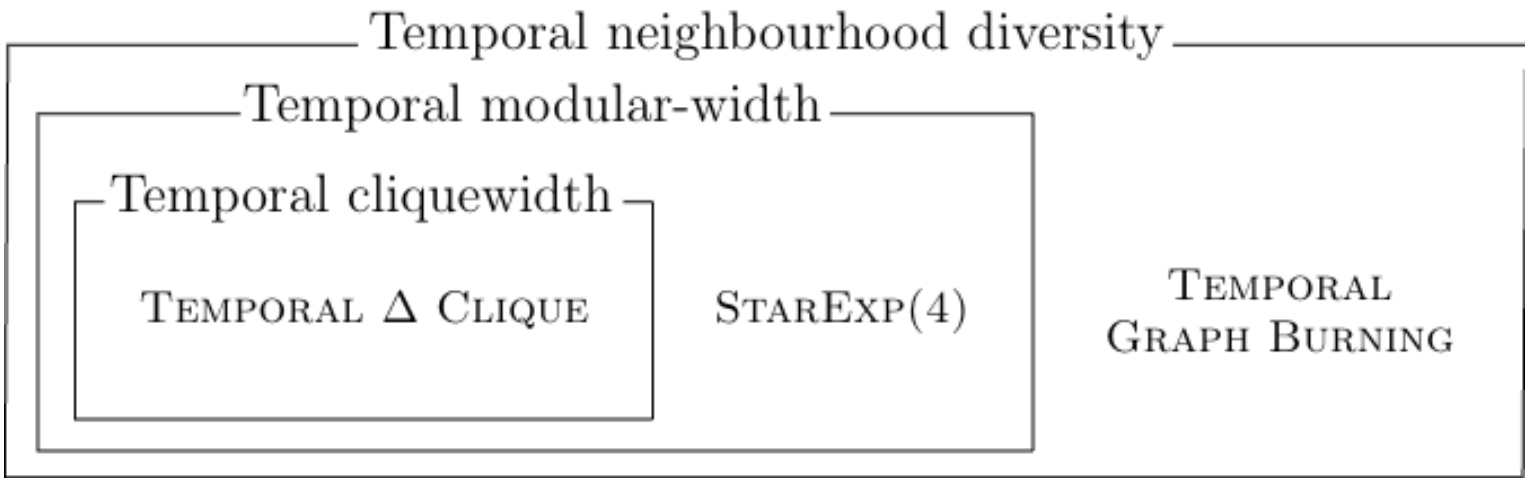
Problem	VIMW	TIMW
Temporal Hamiltonian Path	FPT	FPT
Δ-Temporal Matching	FPT	FPT
Temporal Reachability Edge Deletion	FPT [already known]	FPT
Temporal Firefighter	FPT [already known]	NP-hard

We have characterisations of problems that admit algorithms FPT by TIM and VIM by the requirements on those building blocks.

These parameters are so helpful because they enforce sparseness: most vertices can be ignored most of the time.

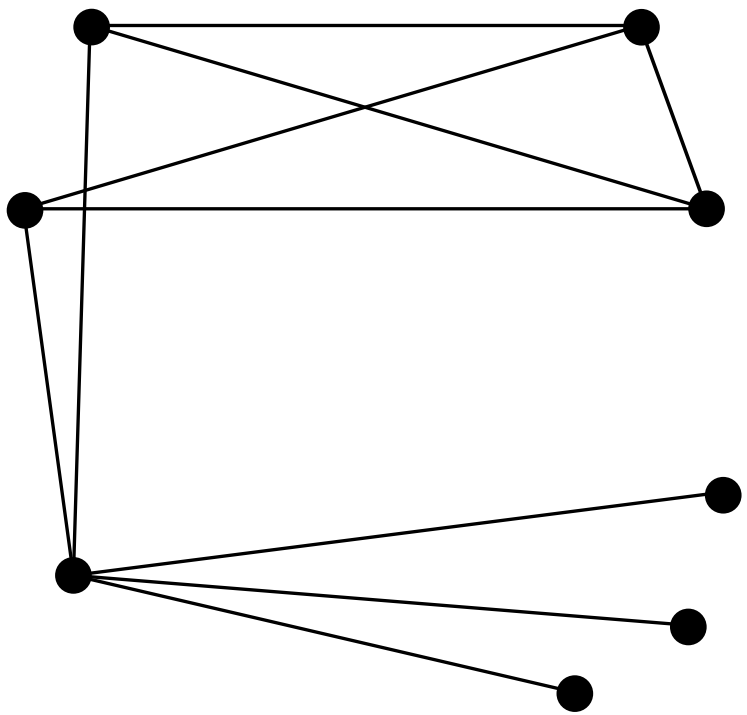
WE WANTED PARAMETERS FOR 'DENSER' TEMPORAL GRAPHS

We have started using a few generalisations from static parameters that allow lots of active edges at a time and give some tractability results.



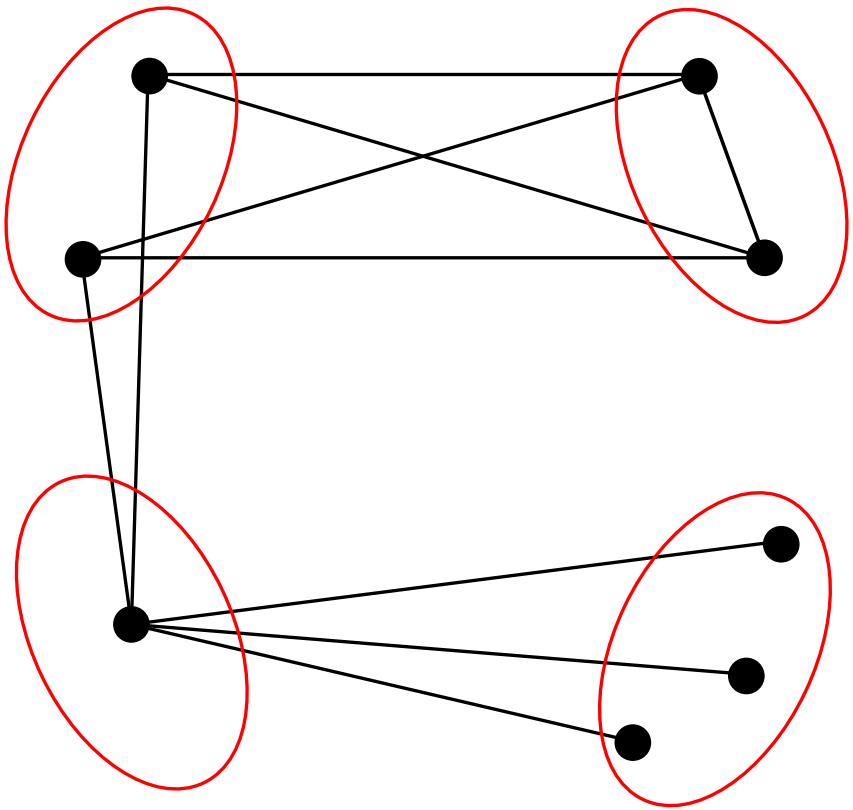
NEIGHBOURHOOD DIVERSITY

The *neighbourhood diversity* of a graph $G = (V, E)$ is the smallest integer k such that V can be partitioned into sets $V_1, \ldots, V_k$ with the property that, if $x, y \in V_i$ for any i then $N(x) \setminus \{y\} = N(y) \setminus \{x\}$.



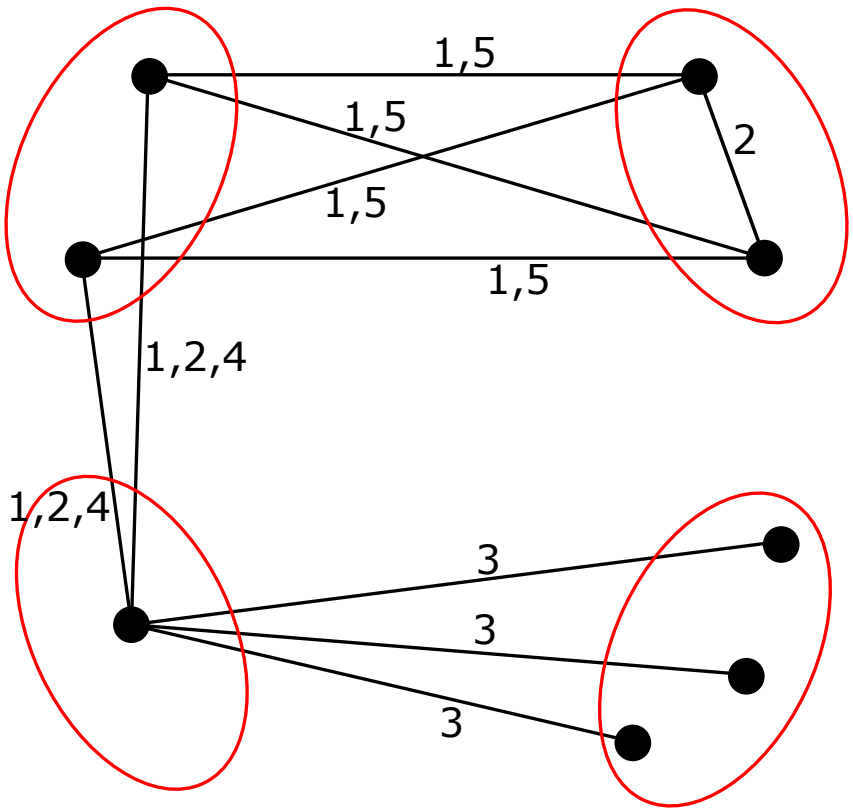
NEIGHBOURHOOD DIVERSITY

The *neighbourhood diversity* of a graph $G = (V, E)$ is the smallest integer k such that V can be partitioned into sets $V_1, \ldots, V_k$ with the property that, if $x, y \in V_i$ for any i then $N(x) \setminus \{y\} = N(y) \setminus \{x\}$.



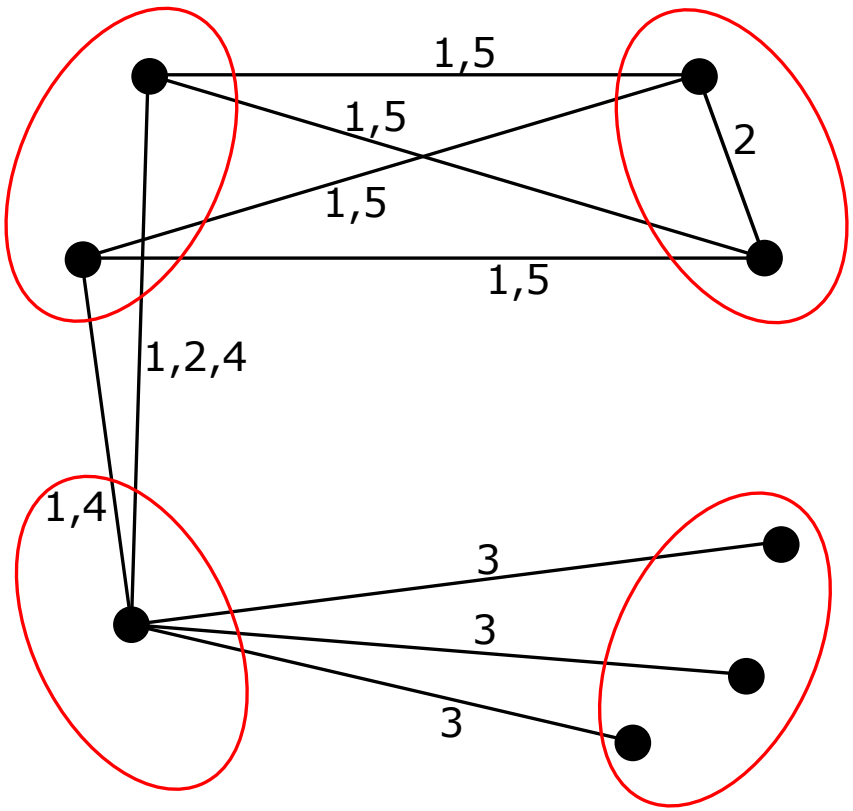
TEMPORAL NEIGHBOURHOOD DIVERSITY

The *temporal neighbourhood diversity* of a temporal graph $(G = (V, E), \lambda)$ is the smallest integer k such that V can be partitioned into sets $V_1, \ldots, V_k$ with the property that, if $x, y \in V_i$ for any i then, for all times t and all vertices $z \notin \{x, y\}$, $t \in \lambda(xz)$ if and only if $t \in \lambda(yz)$.



TEMPORAL NEIGHBOURHOOD DIVERSITY

The *temporal neighbourhood diversity* of a temporal graph $(G = (V, E), \lambda)$ is the smallest integer k such that V can be partitioned into sets $V_1, \ldots, V_k$ with the property that, if $x, y \in V_i$ for any i then, for all times t and all vertices $z \notin \{x, y\}$, $t \in \lambda(xz)$ if and only if $t \in \lambda(yz)$.



Temporal Graph Burning:

1. At time $t = 0$ a fire is placed at a chosen vertex. All other vertices are unburnt.
2. At all times $t \geq 1$, the fire spreads, burning all vertices u adjacent to an already burning vertex v where the edge between u and v is active at time t. Then, another fire is placed at a chosen vertex.
3. This process ends once all vertices are burning.

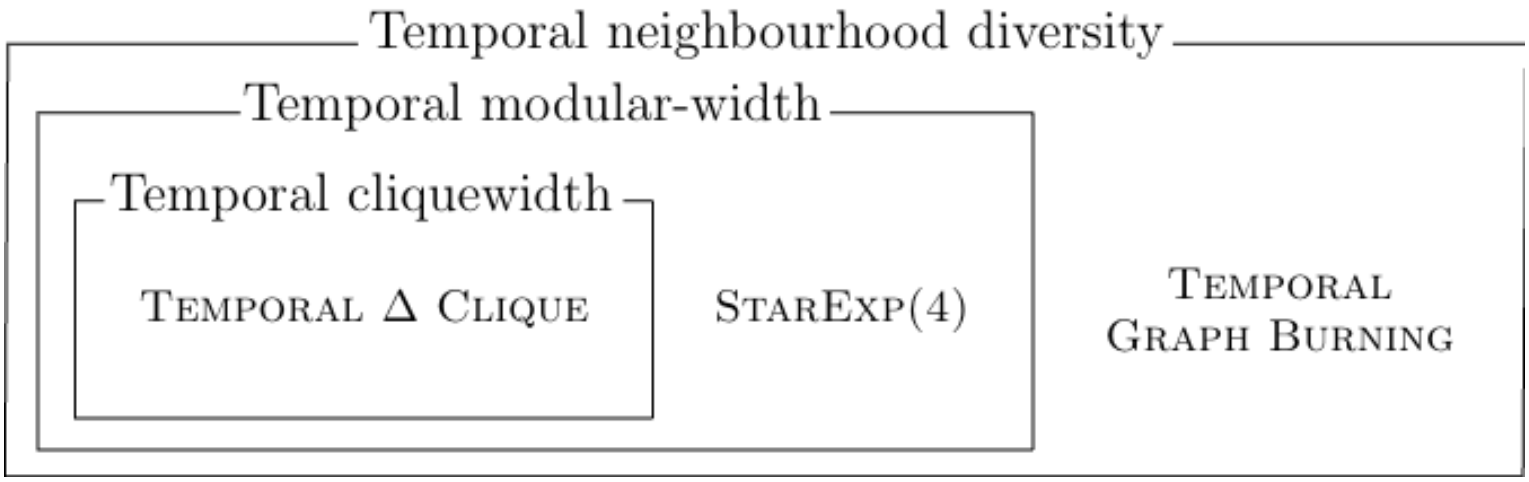
TEMPORAL GRAPH BURNING

Input: A temporal graph (G, λ) and an integer ℓ.

Question: Does there exist a successful burning strategy for (G, λ) of length less than or equal to ℓ?

Theorem

TEMPORAL GRAPH BURNING admits an FPT algorithm parameterised by the temporal neighbourhood diversity of the input graph.



Conjecture

Any temporal graph property expressible in first-order logic admits an FPT algorithm parameterised simultaneously by the temporal cliquewidth and the length of the formula.

Two classical graph games, both about **catching** or **covering** quickly:

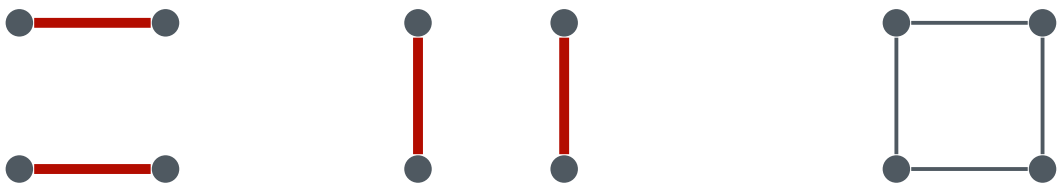
- **Cops and Robbers** (Quilliot 1978; Nowakowski & Winkler 1983): cops and a robber alternate moves along edges; the cop number $c(G)$ is the fewest cops that always catch the robber.
- **Graph Burning** (Bonato, Janssen & Roshanbin 2016): each round a new fire is lit and every fire spreads to all neighbours; the burning number is the fewest rounds to burn everything.

Both have natural temporal versions — and in both, **time cuts both ways**: it can rescue the robber, and it can trap him.

PERIODIC TEMPORAL GRAPHS

A temporal graph with **infinite lifetime but a finite description**:

$\mathcal{G} = (G_0, G_1, G_2, \dots)$ is *periodic with period* p if $G_t = G_{t+p}$ for all t; we write $\mathcal{G} = \langle G_0, \dots, G_{p-1} \rangle$, and the footprint is $G = \bigcup_t G_t$.



G_0 G_1 footprint C_4

Natural whenever the timetable repeats: shift patterns, bus routes, seasonal contact.

A finer variant: give each *edge* its own binary pattern of length ℓ_e, present at time t iff $b_e(t \bmod \ell_e) = 1$. The period is then lcm of the ℓ_e.

COPS AND ROBBERS, PERIODICALLY

The cop moves first and sees the robber; each may stay put or cross an edge **present at that step**.

- Whether one cop wins is decidable in $\mathcal{O}(\mathrm{lcm}(L) \cdot n^3)$ time, and in $\mathcal{O}(\mathrm{lcm}(L) \cdot k \cdot n^{k+2})$ for k cops *(Erlebach & Spooner 2020)*.

- Improved for one cop to $\mathcal{O}(pn^2 + nm)$, with a characterisation of the cop-win periodic graphs *(de Carufel, Flocchini, Santoro & Simard 2023)*.

Note what the parameter is: the period, not the lifetime. Everything is polynomial once the period is bounded, and the blow-up when $\mathrm{lcm}(L)$ is large is the interesting part.

EVEN CYCLES ARE INTERESTING AGAIN

On a static cycle C_n with $n \geq 4$, one cop **never** catches the robber: he simply walks away.

Make the cycle edge-periodic and this changes:

- there are **cop-win** edge-periodic cycles of length $3\,\mathrm{lcm}(L)$;
- but every edge-periodic cycle with $n \geq 2\ell \cdot \mathrm{lcm}(L)$ is **robber-win** *(Erlebach & Spooner 2020)*.

So it is the length measured against the period that decides — a missing edge at the right moment is a wall the robber cannot pass.

How bad can a schedule be?

Our question: fix the footprint, and let an adversary choose the periodic schedule.

$$\hat{c}_{\max}(G) \;=\; \max\{\, c(\mathcal{G}) : \mathcal{G} \text{ periodic with footprint } G \,\}$$

- $\hat{c}_{\max}(G) \leq \mathrm{tw}(G) + 1$, so bounded treewidth still saves you.
- The footprint alone tells you very little: scheduling can cost you many extra cops.

Work in progress with Fred and Georg, begun at Dagstuhl.

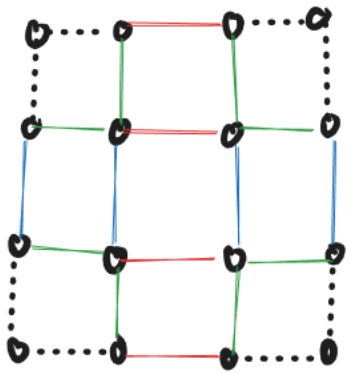
We can force the cop number up on the hypercube by activating one dimension at a time:

Claim

$\hat{c}_{\max}(Q_k) \geq k$.

(Believed, under internal review — the schedule activates dimension i at step i, and the robber keeps a dimension free of each cop.)

We expected equality. The computations say otherwise: $d = 6$ needs 8 cops, $d = 7$ needs 12.



The same trick on $P_4 \square P_4$: period 9, and we think 3 cops are needed.

SUMMARY AND THANKS!

In summary:
There are a variety of approaches to parameters for temporal graphs.
Next up:

- Find more problems that are tractable parameterised by these parameters
- Is there a Courcelle-style metatheorem for temporal cliquewidth?
- Investigate the values of these parameters on real-world temporal networks

TEMPORAL C-CLOSEDNESS

Now for something more inspired by the complex networks community.

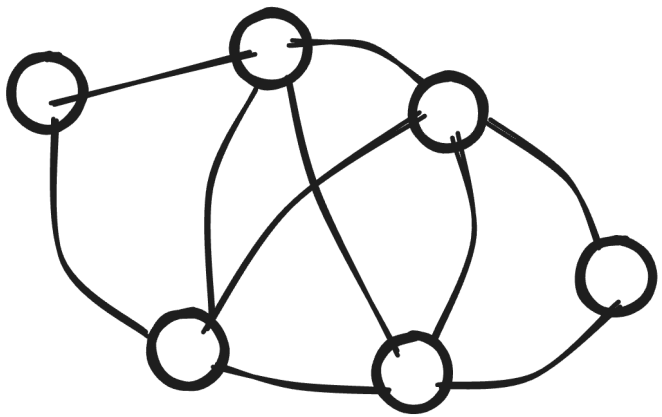
A temporal graph parameter that measures whether vertices that share neighbours over some small timespan are also themselves adjacent.

If a graph has low value of this parameter, and can't change too much from one time to the next, then we can efficiently enumerate its temporal cliques.

Some graphs from data have low-ish values for this parameter, or are close to doing so.

c-CLOSEDNESS

A static graph is c-closed if every pair of vertices that share at least c neighbours are adjacent.



This bounds the number of maximal cliques, and helps with various algorithmic questions

Fox, J.; Roughgarden, T.; Seshadhri, C.; Wei, F.; and Wein, N. 2018. Finding Cliques in Social Networks: A New Distribution-Free Model. In *ICALP*, volume 107 of *LIPIcs*, 55:1–55:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.

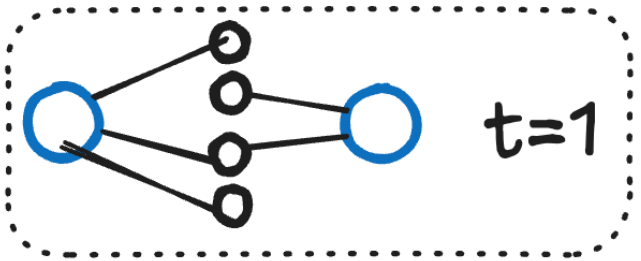
WHAT DOES THIS MEAN FOR A TEMPORAL GRAPH?

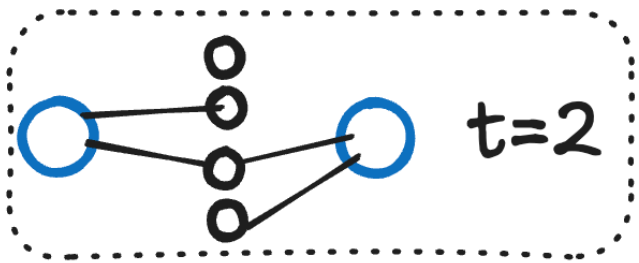
What does it mean to share neighbours?

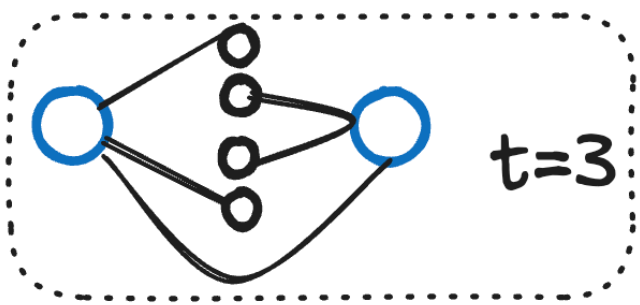
- Ever?
- In some time window?
- Consistently?

When would they then be adjacent?

- Ever?
- When they share neighbours?
- After they share neighbours?
- Before they share neighbours?
- Always?







WE HAVE PICKED A MULTI-PARAMETER DEFINITION

A temporal graph (G, λ) is $(\Delta_0, \Delta_1, \Delta_2, c)$-closed when:

- for every two distinct vertices $u, v \in V(G)$ and
- any time-interval $[a, b]$ of length at most $\Delta_1 + 1$ (i.e., $b - a \leq \Delta_1$) with $a \geq 1 + \Delta_0$ and $b \leq \Lambda_{\mathcal{G}} - \Delta_2$,

if

- u and v have at least c common neighbors during $[a, b]$,

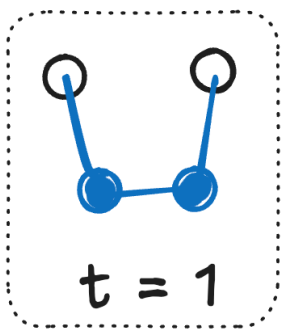
then

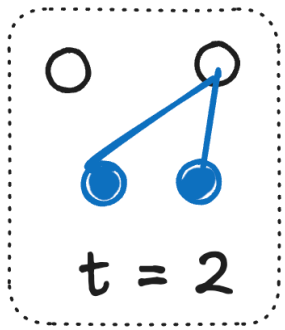
- u and v are adjacent to each other during $[a - \Delta_0, b + \Delta_2]$

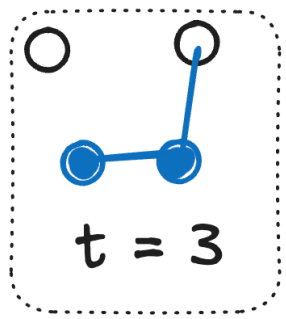
before after

$(\Delta_0, \Delta_1, \Delta_2, c)$-closed

span







(1,1,0,1)-closed
(1,1,1,1)-closed
(0,1,1,1)-closed
not (0,1,0,1)-closed

The black vertices on top are not $(x, 2, y, 1)$—closed for any x, y

WHAT DOES THIS LET YOU DO?

In the static case, closure lets you enumerate maximal cliques efficiently.

That's our aim here, but we need something else to bound the number of maximal temporal cliques:

local stability

BUT WAIT - WE NEED MORE!

Definition (locally η-unstable graphs)

For a non-negative integer η, a temporal graph $\mathcal{G} = (G, \lambda)$ is locally η-unstable if $\max\{|N_t(v) \setminus N_{t+1}(v)|, |N_{t+1}(v) \setminus N_t(v)|\} \leq \eta$ for every vertex $v \in V(G)$ and every time-step $t \leq \Lambda - 1$.

Intuition: neighbourhoods can change by at most η vertices each time step.

I've lied to you slightly again...

DON'T READ THIS SLIDE. YOU DON"T WANT TO KNOW.

What we need is actually a little weaker than this - it's that the common neighbourhoods of vertices can't change too fast:

Definition (pairwise η-unstable graphs)

For a non-negative integer η, a temporal graph $\mathcal{G} = (G, \lambda)$ is pairwise η-unstable if for any two distinct vertices u and v and any time-interval $[a, b]$ and non-negative integers ℓ, ℓ' such that $[a - \ell, b + \ell'] \subseteq [1, \Lambda_{\mathcal{G}}]$, we have $|CN_{[a-\ell,b+\ell']}(u, v) \setminus CN_{[a,b]}(u, v)| \leq \eta(\ell + \ell')$.

With all of this bounding machinery:

You can enumerate cliques efficiently parameterised by the triadic closure parameters plus the stability parameter:

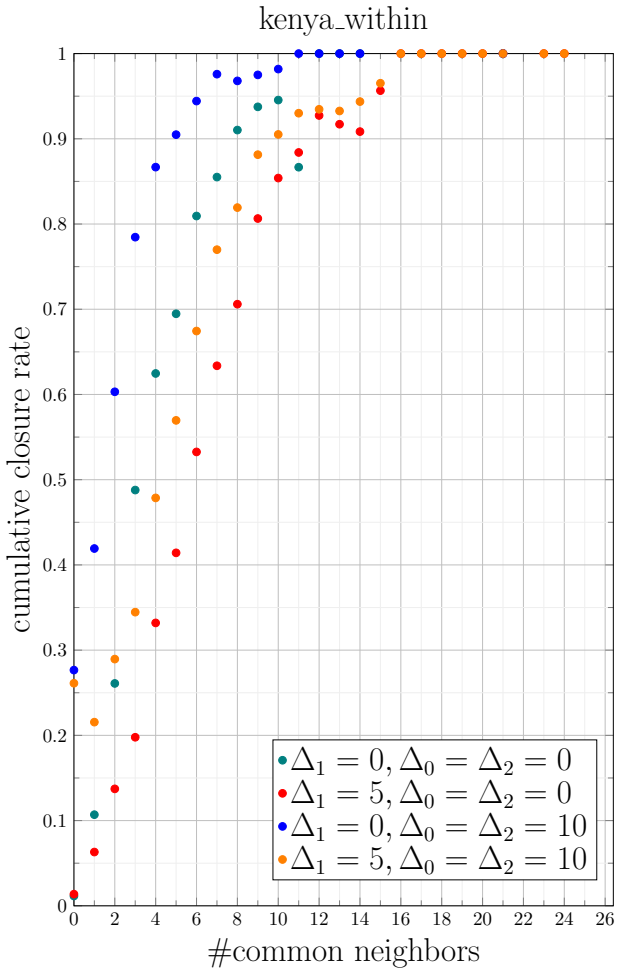
$$\mathcal{O}(2^{c-1+2\eta(\Delta+1-\Delta_1)} \cdot n^2 \cdot m \cdot \Lambda)$$

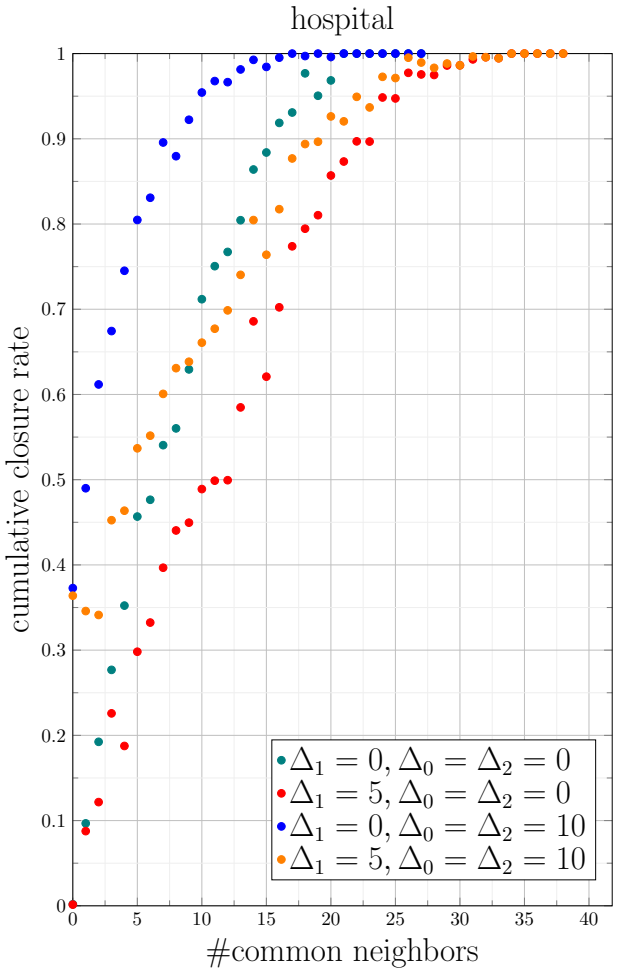
or

$$\mathcal{O}((\text{exp in the params})(\text{poly in input size}))$$

DO DATA-DERIVED GRAPHS HAVE THESE PROPERTIES?

How much does this occur in these particular graphs?





GRAPHS?

Somewhat.

We looked at some SocioPatterns networks, and found that these parameters vary widely by network.

For example, for two different workplace networks, we found:

- One with 95 nodes, instabilities in the 70s
- One with 217 nodes, instabilities in the 20s

The picture is similar for closedness measures.

- When are these useful?
- What is the right time step?
- Where do these fit in the landscape of temporal graph parameters?